PRODUCT REQUIREMENTS DOCUMENT

FinPay Core Engine

RESTful Web API | FinTech & Digital Banking

Project Name	FinPay Core Engine (RESTful Web API)
Domain	FinTech / Digital Banking — similar in concept to SadaPay, NayaPay, or Wise backends
Target Architecture	Clean / Layered Architecture using .NET 8/9, Entity Framework Core, and SQL Server

PART 1 — PRODUCT REQUIREMENTS DOCUMENT (PRD)

1.1 Product Vision

FinPay is a high-integrity backend transaction engine that allows users to open digital wallets, deposit and withdraw funds, and transfer money to one another in real time through peer-to-peer (P2P) payments. The system prioritizes **zero data loss**, **concurrency protection** (preventing duplicate or overlapping transfers), and a **double-entry ledger** that records and accounts for every movement of funds.

1.2 User Roles and Personas

1. Standard User (Role: Customer)

- Can register for and log in to an account.
- Can view their wallet balance and transaction statements/history.
- Can send money to another user using the recipient's wallet number or phone number (P2P transfer).

2. System Administrator (Role: Admin)

- Can freeze or block any wallet flagged as suspicious.
- Can view all system transactions and generate audit reports for high-value transactions.
- Can upgrade a user's KYC tier from Tier 1 (Basic) to Tier 2 (Verified).

1.3 Core Business Rules

These rules are intended to be implemented in C# using object-oriented programming (OOP), domain models, and service-layer logic.

Rule 1 — Account Tiers and Limits (Inheritance and Polymorphism)

Wallet Type	Daily Transfer Limit	Fees / Charges
Basic Wallet (Level 1)	PKR 50,000	PKR 0 fee on each P2P transfer. ATM/bank withdrawals incur a 1.5% fee.
Merchant / Business Wallet (Level 2)	PKR 1,000,000	A 0.5% service tax is deducted from each incoming payment.

Rule 2 — No Negative Balance (Encapsulation)

A wallet balance must never fall below zero. The balance property must not be directly modifiable from outside the wallet/domain logic; changes must occur through controlled operations that validate the transaction.

Rule 3 — Block Self-Transfers

A user must not be able to transfer funds to their own wallet number.

Rule 4 — Atomic Transfers (ACID Transactions)

When User A transfers PKR 5,000 to User B, the system must perform all of the following as one atomic operation:

- Debit PKR 5,000 from User A's wallet balance.
- Credit PKR 5,000 to User B's wallet balance.
- Insert two entries into the ledger: one **Debit** entry and one **Credit** entry.

If a database error occurs while applying the recipient credit or inserting either ledger entry, the entire operation must be rolled back, including the original debit. No partial transfer may be committed.